In God we trust

نام و نام خانوادگی : مهدیس میرزایی

شماره دانشجویی : ۸۱۰۱۰۲۵۵۰

پروژه : پروژه شماره ۴

بخش یک :

۱ - ابتدا مپ‌ستی ایجاد می‌کنیم که در سطر اول حروف و در سطر دوم باینری آن قرار دارد و به صورت زیر خواهد بود :

```
%% 1-Creating the mapset
NumberOfChar = 32;
mapset = cell(2,NumberOfChar);
Alphabet = 'abcdefghijklmnopqrstuvwxyz .,!;"';
for i = 1:NumberOfChar
    mapset{1,i} = Alphabet(i);
    mapset{2,i} = dec2bin(i-1,5);
end
```

و در متلب به صورت زیر بود :

```
mapset =

  2×32 cell array

  Columns 1 through 13

    {'a'}    {'b'}    {'c'}    {'d'}    {'e'}    {'f'}    {'g'}    {'h'}    {'i'}    {'j'}    {'k'}    {'l'}    {'m'}
    {'00000'}    {'00001'}    {'00010'}    {'00011'}    {'00100'}    {'00101'}    {'00110'}    {'00111'}    {'01000'}    {'01001'}    {'01010'}    {'01011'}    {'01100'}

  Columns 14 through 26

    {'n'}    {'o'}    {'p'}    {'q'}    {'r'}    {'s'}    {'t'}    {'u'}    {'v'}    {'w'}    {'x'}    {'y'}    {'z'}
    {'01101'}    {'01110'}    {'01111'}    {'10000'}    {'10001'}    {'10010'}    {'10011'}    {'10100'}    {'10101'}    {'10110'}    {'10111'}    {'11000'}    {'11001'}

  Columns 27 through 32

    {' '}    {'.'}    {','}    {'!'}    {';'}    {'"'}
    {'11010'}    {'11011'}    {'11100'}    {'11101'}    {'11110'}    {'11111'}

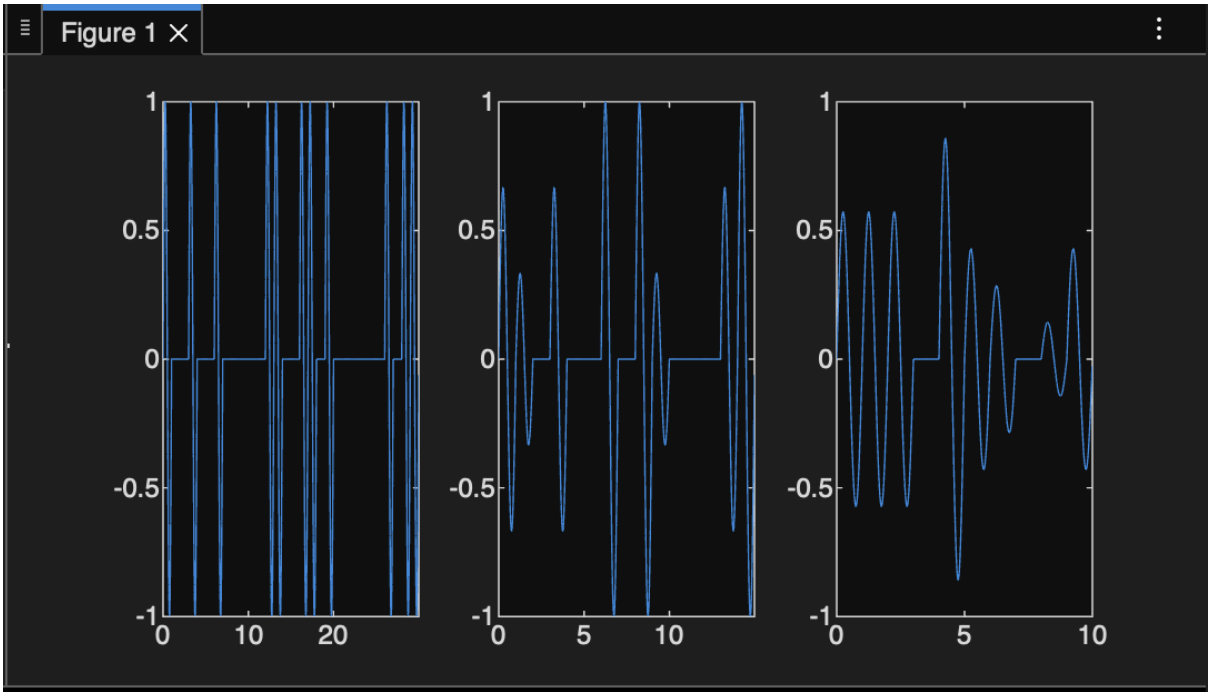
>>
```

۲- ابتدا با استفاده از Mapset طراحی‌شده، تک‌تک کاراکترهای پیام را به معادل باینری تبدیل و در یک آرایه یکپارچه ذخیره می‌کنیم. سپس این رشته بیت‌ها را بر اساس مقدار «بیت ریت» دسته‌بندی کرده و برای هر دسته، عدد دسیمال معادل را محاسبه می‌نماییم. این عدد به عنوان ضریب دامنه در نظر گرفته می‌شود و در نهایت، با اعمال آن در فرکانس نمونه‌برداری، موج سینوسی متناظر تولید می‌گردد.

```
function encoded_wave = coding_amp(input_str, R, char_map)
    Fs = 100;
    encoded_wave = [];
    MapSize = 32;
    L = strlength(input_str);
    temp_bins = cell(1, L);
    for i = 1:L
        this_char = input_str(i);
        for j = 1:MapSize
            if this_char == char_map{1,j}
                temp_bins{i} = char_map{2,j};
            end
        end
    end
    bit_stream = cell2mat(temp_bins);
    total_bits = length(bit_stream);
    for idx = 1:R:total_bits
        if idx + R - 1 > total_bits
            break;
        end
        current_bits = bit_stream(idx : idx + R - 1);
        dec_val = bin2dec(current_bits);
        amp_coeff = dec_val / ((2^R) - 1);
        t_start = idx / R;
        time_vec = t_start : (1/Fs) : t_start + 1 - (1/Fs);
        wave_chunk = amp_coeff * sin(2 * pi * (time_vec - t_start));
        encoded_wave = [encoded_wave wave_chunk];
    end
end
```

۳- حال طبق خواسته این بخش خروجی تابع coding_amp را برای پیام signal با سرعت ارسال اطلاعات یک، دو و سه بیت بر ثانیه به صورت جداگانه رسم می‌کنیم.

```
Fs = 100;
txt_msg = 'signal';
enc_sig = cell(1,3);
figure
for k=1:3
    enc_sig{k} = coding_amp(txt_msg,k, mapset);
    time_vec = 0:(1/Fs):(length(enc_sig{k})/Fs)-(1/Fs);
    subplot(1,3,k);
    plot(time_vec,enc_sig{k})
end
```



۴- در این مرحله، معکوس کدگذاری اجرا می‌شود. ابتدا سیگنال دریافتی به بازه‌های زمانی یک‌ثانیه‌ای تقسیم شده و Correlation هر بازه با سیگنال مرجع سینوسی محاسبه می‌گردد تا ضریب دامنه به دست بیاید. این ضرایب به اعداد دسیمال و سپس به رشته‌های باینری تبدیل می‌شوند. در نهایت، با اجماع تمام بیت‌ها و دسته‌بندی آن‌ها در گروه‌های ۵ تایی، و با استفاده از Mapset، کاراکترهای متناظر شناسایی شده و پیام اصلی بازیابی میشود.

```
function decoded_message = decoding_amp(coded_sig,bit_rate, mapset)
    fs = 100;
    decoded_message = [];
    bin_message = [];
    thresholds=[];
    for k=1:(2^bit_rate)-1
        thresholds(k) = ((2*k)-1)/(2*((2^bit_rate)-1));
    end
    t = 0:(1/fs):1-(1/fs);
    ref = 2*sin(2*pi*t);
    for j=0:(length(coded_sig)/fs)-1
        to_check = coded_sig((j*fs)+1:((j+1)*fs));
        corr = 0.01*sum(to_check.*ref);
        initial_corr = thresholds(1);
        for t=1:length(thresholds)
            if corr<=thresholds(t)
                corr = ((initial_corr*2*((2^bit_rate)-1))-t)/((2^bit_rate)-1);
                break;
            elseif t==length(thresholds)
                corr = 1;
            else
                initial_corr = thresholds(t+1);
            end
        end
        bin_message = [bin_message dec2bin(round(corr*((2^bit_rate)-1)),bit_rate)];
    end
    for p = 1:5:length(bin_message)-4
        message_to_check = bin_message(p:p+4);
        message_to_check = mapset{1,bin2dec(message_to_check)+1};
        decoded_message = [decoded_message message_to_check];
    end
end
```

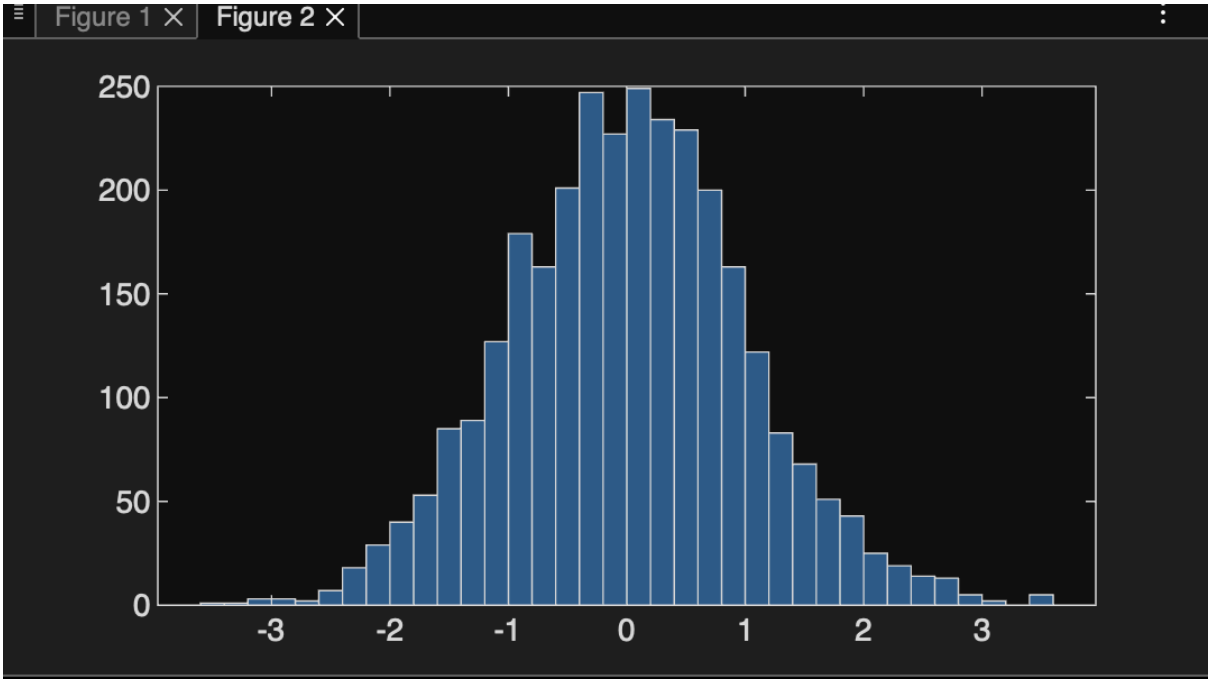
و حال decode میکنیم و داریم :

```
%% 4
rec_str = cell(1,3);
for k=1:3
    rec_str{k} = decoding_amp(enc_sig{k},k);
    fprintf("Message number %d with bitrate = %d and the message = %s \n", k, k,
rec_str{k});
end
```

```
Message number 1 with bitrate = 1 and the message = signal
Message number 2 with bitrate = 2 and the message = signal
Message number 3 with bitrate = 3 and the message = signal
>>
```

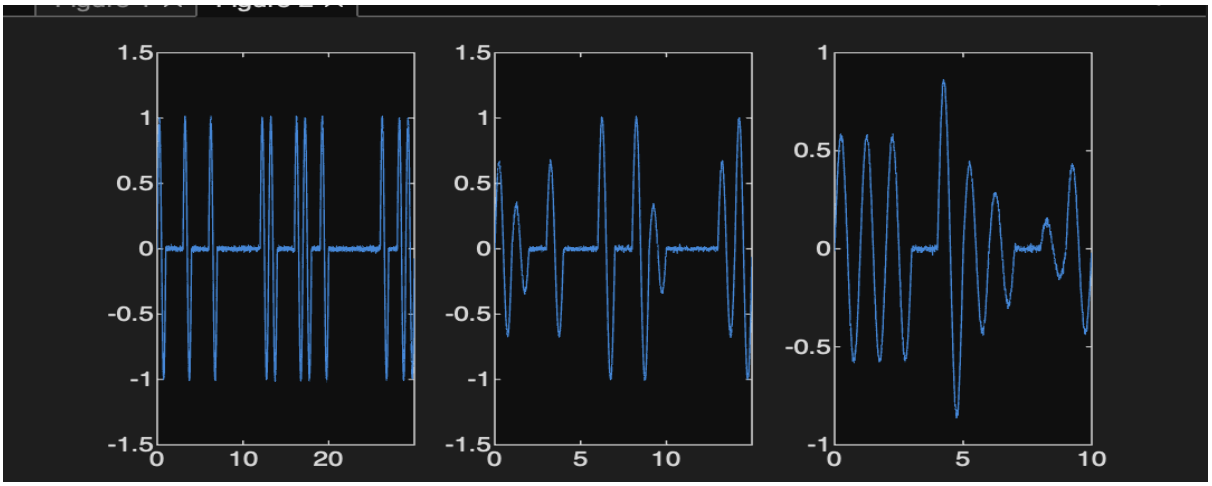
۵- در این قسمت نویزی با استفاده از توزیع گوسی میخواهیم اضافه کنیم. همانطور که مشخص است میانگین 0 و واریانس 1 است.

```
%% 5
g_noise = randn(1,3000);
figure
histogram(g_noise)
mu = mean(g_noise)
sig2 = var(g_noise)
```



۶- چون واریانس نشان می‌دهد که نویز چقدر تغییرات شدید و پراکنده دارد، وقتی مقدار آن زیاد می‌شود، نوسانات نویز آنقدر بالا می‌رود که سیگنال اصلی را مخدوش می‌کند. در نتیجه، تشخیص سطوح دامنه به درستی انجام نشده و پیام پنهان‌شده قابل خواندن نخواهد بود.

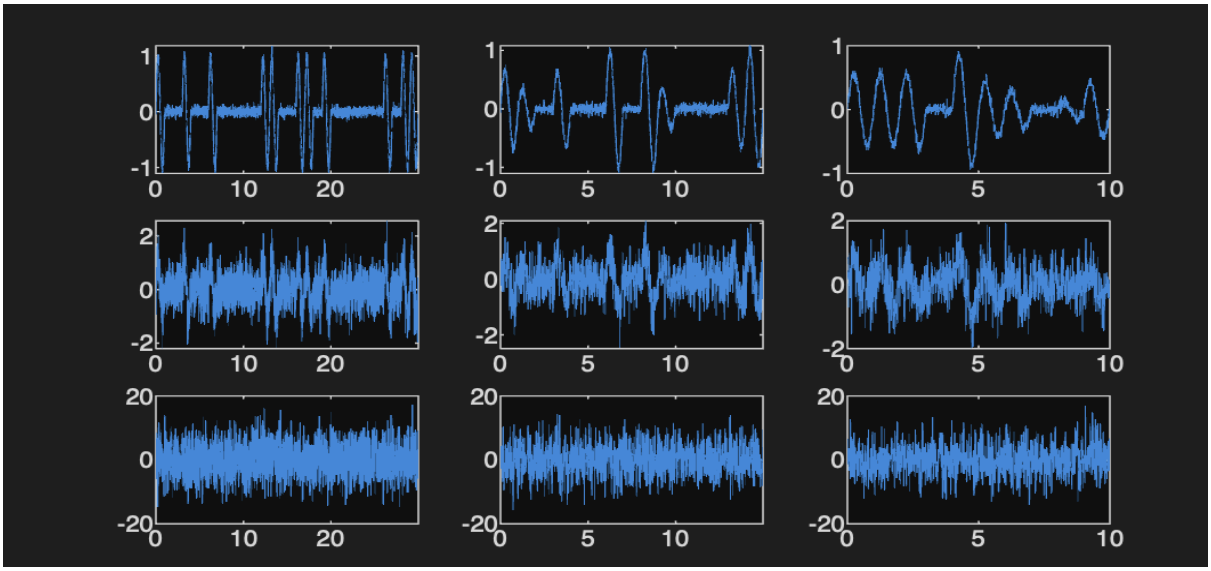
```
%% 6
amp = 0.01;
sig_w_noise = cell(1,3);
for k=1:3
    curr_noise = amp * randn(1, length(enc_sig{k}));
    sig_w_noise{k} = enc_sig{k} + curr_noise;
    time_vec = 0:(1/Fs):(length(sig_w_noise{k})/Fs)-(1/Fs);
    subplot(1,3,k);
    plot(time_vec,sig_w_noise{k})
    dec_msg = decoding_amp(sig_w_noise{k},k, mapset);
    fprintf("Message number %d with bitrate = %d and the message = %s \n", k, k,
dec_msg);
end
```



۷- هرچه سرعت ارسال بالاتر می‌رود، آسیب‌پذیری سیگنال نسبت به نویز بیشتر شده و احتمال اینکه پیام اشتباه دریافت شود افزایش می‌یابد. همچنین در این قسمت با تست کردن سه مقدار مختلف برای انحراف معیار، دیدیم که هرچه این مقدار بیشتر باشد، نویز اثر مخرب‌تری روی سیگنال می‌گذارد و تشخیص پیام سخت‌تر می‌شود.

```
%% 7
Amp_Levels = [0.05, 0.5, 5];
figure
for j = 1:3
    curr_amp = Amp_Levels(j);
    fprintf('Standard Deviation = %d\n', curr_amp);
    for k=1:3
        added_noise = curr_amp * randn(1, length(enc_sig{k}));
        sig_w_noise{k} = enc_sig{k} + added_noise;
        time_vec = 0:(1/Fs):(length(sig_w_noise{k})/Fs)-(1/Fs);
        subplot(3,3,3*(j-1) + k);
        plot(time_vec,sig_w_noise{k})
        dec_msg = decoding_amp(sig_w_noise{k},k, mapset);
        fprintf("Message number %d with bitrate = %d and the message = %s \n", k, k,
dec_msg);
    end
end
```

و طبق خروجی در قسمت‌های آخر دیگر غلط است.

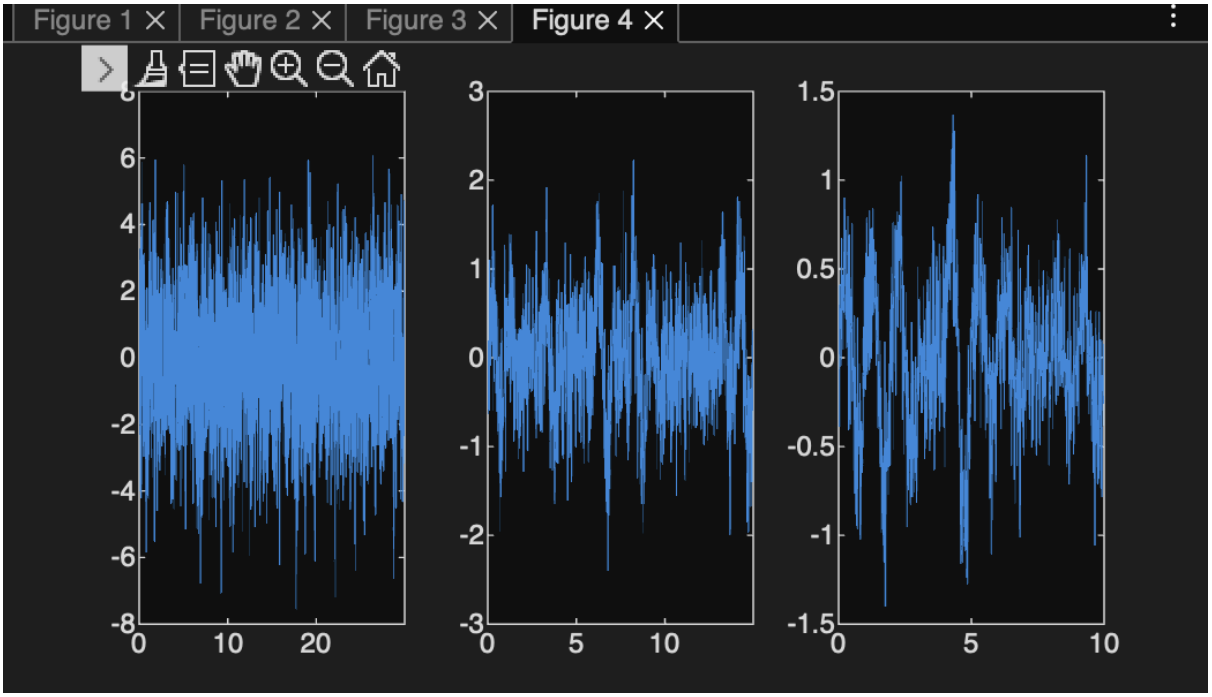


```
Standard Deviation = 5.000000e-02
Message number 1 with bitrate = 1 and the message = signal
Message number 2 with bitrate = 2 and the message = signal
Message number 3 with bitrate = 3 and the message = signal
Standard Deviation = 5.000000e-01
Message number 1 with bitrate = 1 and the message = signal
Message number 2 with bitrate = 2 and the message = signal
Message number 3 with bitrate = 3 and the message = oignsd
Standard Deviation = 5
Message number 1 with bitrate = 1 and the message = woenij
Message number 2 with bitrate = 2 and the message = g,wmil
Message number 3 with bitrate = 3 and the message = ,kpeqh
>>
```

۸- چون نویز به صورت رندوم تولید می‌شود، گاهی اوقات مقادیر سیگنال را درست روی مرز تغییر وضعیت قرار می‌دهد. به همین دلیل، با اجرای مجدد کد ممکن است همیشه نتیجه یکسان نباشد؛ یعنی یا پیام عیناً به دست می‌آید یا ممکن است تفاوت خیلی کمی (مثلاً در حد یک حرف) با پیام اصلی داشته باشد.

```
%% 8
Test_Levels = [2, 0.5, 0.3];
figure
for k=1:3
    lim_amp = Test_Levels(k);
    lim_noise = lim_amp * randn(1, length(enc_sig{k}));
    sig_w_noise{k} = enc_sig{k} + lim_noise;
    time_vec = 0:(1/Fs):(length(sig_w_noise{k})/Fs)-(1/Fs);
    subplot(1, 3, k);
    plot(time_vec,sig_w_noise{k})
    dec_msg = decoding_amp(sig_w_noise{k},k, mapset);
    fprintf("Message number %d with bitrate = %d and the message = %s \n", k, k,
dec_msg);
end
```

و خروجی به صورت زیر شد :




```
Standard Deviation = 5
Message number 1 with bitrate = 1 and the message = wiejel
Message number 2 with bitrate = 2 and the message = ;nemep
Message number 3 with bitrate = 3 and the message = vcf ab
Message number 1 with bitrate = 1 and the message = signal
Message number 2 with bitrate = 2 and the message = signal
Message number 3 with bitrate = 3 and the message = signal
>>
```

۹- با افزایش توان فرستنده و دامنه سیگنال ارسالی، Thresholds افزایش می‌یابد. این امر موجب بهبود نسبت سیگنال به نویز (SNR) شده و قابلیت تفکیک سیگنال اصلی از نویز را ارتقا می‌دهد. در نتیجه، سیستم در برابر نویزهای محیطی مقاوم‌تر شده و احتمال بروز خطای تشخیص در گیرنده به طرز چشمگیری کاهش می‌یابد.

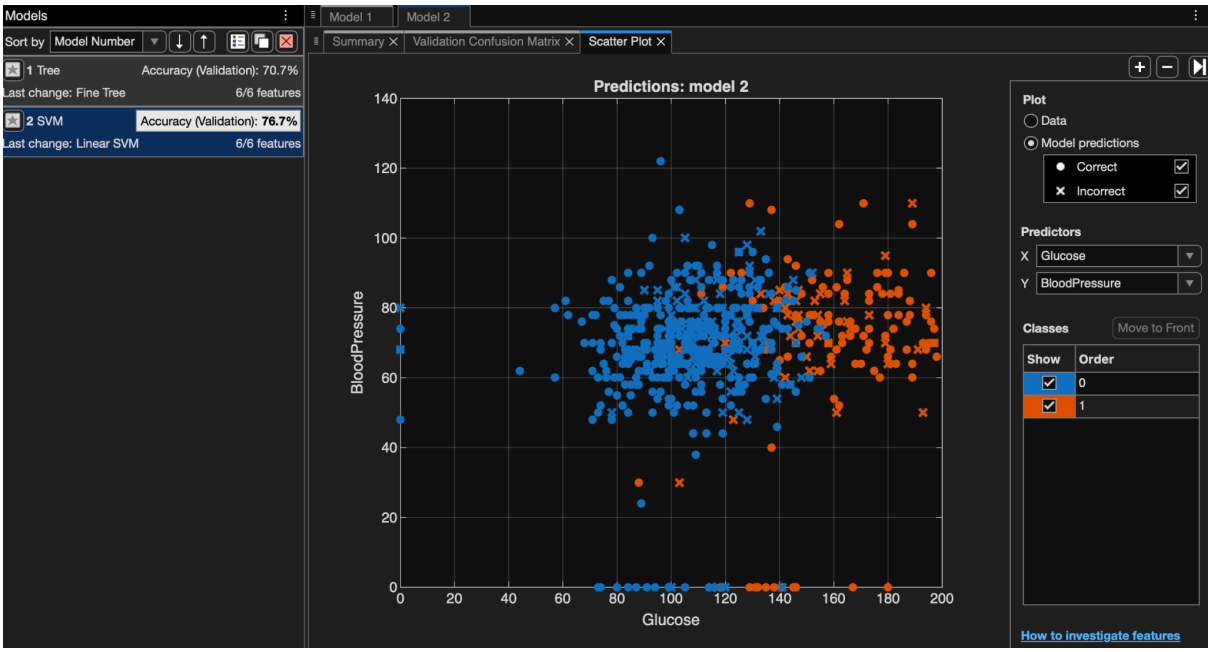
۱۰- در صورت نبودن نویز در سیستم bit rate میتواند به طور تئوری به بینهایت نزدیک شود. به عبارت دیگر، در یک محیط بدون نویز، محدودیتی برای افزایش bit rate وجود ندارد. البته، در عمل موارد دیگری مانند محدودیتهای سخت افزاری، پهنای باند کانال و محدودیت های فیزیکی سیستم ها تاثیرگذار خواهند بود.

۱۱- ضرب کردن سیگنال دریافتی در یک عدد ثابت (گین گیرنده)، تاثیری در بهبود عملکرد در برابر نویز ندارد. زیرا این عملیات هم سیگنال پیام و هم نویز سوار شده بر آن را به یک نسبت تقویت می‌کند. در نتیجه نسبت سیگنال به نویز (SNR) ثابت باقی می‌ماند. برای کاهش اثر نویز، تقویت باید در مبدا (فرستنده) و قبل از اضافه شدن نویز کانال صورت گیرد.

۱۲- سرعت اینترنت خانگی (ADSL) معمولاً در بازه ۸ تا ۱۶ مگابیت بر ثانیه قرار دارد. نرخ ارسال تنها ۱ تا ۳ بیت بر ثانیه بود که نشان‌دهنده اختلاف فاحشی (حدود ۶ تا ۷ مرتبه بزرگی) با سیستم‌های عملیاتی است. هدف از این شبیه‌سازی با سرعت پایین، درک بهتر مفاهیم پایه مدولاسیون و اثر نویز بود، نه شبیه‌سازی مقیاس واقعی

بخش دو :

۱- در این بخش خروجی ۷۶.۷ درصد بود و scatter plot در زیر نمایش داده شده است.



۲- طبق خروجی هر یک از فیچرها جدول زیر را ایجاد کرده ایم :

فیچر	دقت
Glucose	۷۴.۳
Blood Pressure	۶۵.۳
Skin Thickness	۶۵.۳
Insulin	۶۵.۳
BMI	۶۵.۳
Age	۶۵.۵

پس گلوکز بیشترین تاثیر را دارد.

۳-ابتدا مدل را ذخیره می‌کنیم

```
>> load("/MATLAB Drive/trainedModel.mat")
>> trainedModel

trainedModel =

  struct with fields:

         predictFcn: @(x)exportableModel.predictFcn(predictorExtractionFcn(x))
    RequiredVariables: {'Glucose'  'BloodPressure'  'SkinThickness'  'Insulin'  'BMI'  'Age'}
    ClassificationSVM: [1×1 ClassificationSVM]
                About: 'This struct is a trained model exported from Classification Learner R2025b.'
          HowToPredict: 'To make predictions on a new table, T, use: ↵  [yfit,scores] = c.predictFcn(T) ↵replace 'c' with the name of the variable that is this struct, e.g.

>>
```

```
%% 3
train_dataset = readtable('diabetes-training.csv');
actual_train_labels = train_dataset.label;
predicted_train_labels = trainedModel.predictFcn(train_dataset);
train_score = sum(predicted_train_labels == actual_train_labels) /
numel(actual_train_labels) * 100;
fprintf('Training Accuracy: %.2f%%\n', train_score);
```

و طبق خروجی به دقت ۷۷.۵ رسیدیم.

```
>> main2
Training Accuracy: 77.50%
>>
```

۴- حال برای دیتاهای فایل گفته شده با استفاده از مدل داریم :

```
%% 4
val_dataset = readtable('diabetes-validation.csv');
actual_val_labels = val_dataset.label;
predicted_val_labels = trainedModel.predictFcn(val_dataset);
val_score = sum(predicted_val_labels == actual_val_labels) / numel(actual_val_labels)
* 100;
fprintf('Validation Accuracy: %.2f%%\n', val_score);
```

و به خروجی زیر رسیدیم.

```
>> main2
Training Accuracy: 77.50%
Validation Accuracy: 78.00%
>>
```